

Assignment 3: Deep Reinforcement Learning on HighwayEnv

Sanjay Gupta,
(s347gupta@uwaterloo.ca)

University of Waterloo, Waterloo, Canada

This report presents the implementation and evaluation of various Deep Reinforcement Learning (DRL) algorithms using the HighwayEnv framework created by the Farama Foundation. The focus is on training agents in different environments, including highway, intersection, and racetrack scenarios. The report details the algorithms used, parameter tuning strategies, and empirical results obtained from the experiments.

0 Introduction

The highway environment is a popular domain for evaluating Deep Reinforcement Learning (DRL) algorithms, providing a realistic simulation of driving scenarios.

Throughout the experiments, we employed core DRL algorithms, namely DQN, PPO, SAC, and TRPO from the Stable Baselines3 library. We use TRPO for the racetrack environment since it only supports continuous action spaces out of the box – we experienced difficulties when updating the racetrack environment to support discrete meta-actions (refer to table 1 for details). Note that we don't pass series of frames to the agent, but rather use the state information provided by the environment, which includes the agent's position, velocity, and other relevant features including those of other vehicles in the environment.

1 Environment Domains

The experiments were conducted using the HighwayEnvMod framework, which is a modified version of the original HighwayEnv, with updated reward structures and environment configurations to train the model to learn more interesting behaviors.

- **Highway Environment: 'highway-fast-v0'** The agent learns to drive on a highway, avoiding collisions and maintaining speed. The environment is updated to run simulations and train the agent faster – compared to the original highway-v0; has a discrete action space.
- **Intersection Environment:** The agent learns to navigate through intersections, managing traffic and avoiding collisions. The destination can be set within the environment configuration, but for our experiments, the

agent tries to navigate to 'o1' (the first destination). API provides two versions of the intersection environment: 'intersection-v0' (discrete action space) and 'intersection-v1' (continuous action space).

- **Racetrack Environment:** The agent learns to navigate a racetrack, focusing on speed and control while avoiding collisions.

2 Algorithms

We implemented the following algorithms from the Stable Baselines3 and Stable Baselines3 Contrib libraries:

- **DQN (Deep Q-Network):** A value-based off-policy algorithm – uses a neural network to approximate the Q-value function.
- **PPO (Proximal Policy Optimization):** An on-policy-based algorithm that optimizes the policy directly using a surrogate objective function.
- **SAC (Soft Actor-Critic):** An off-policy actor-critic algorithm that uses a stochastic policy and incorporates entropy regularization to encourage exploration.
- **TRPO (Trust Region Policy Optimization):** An on-policy optimization algorithm that ensures updates to the policy are within a trust region, improving stability and performance.

3 Parameter Tuning

For each algorithm, we defined a separate notebook and conda environments (with the dependencies for each environment being the same) to run the hyperparameter tuning process for each algorithm on independent cores. We uti-

Algorithm	Highway	Intersection	Racetrack
DQN	highway-fast-v0	intersection-v0	—
PPO	highway-better-v1	intersection-v{0,1}	racetrack-v0
SAC	highway-better-v1	intersection-v1	racetrack-v0
TRPO	—	—	racetrack-v0

Table 1. Algorithms tested on different base environments. The table above shows the environments used for training each algorithm. DQN always uses discrete action space; PPO can work on discrete as well as continuous; SAC only works on continuous action space.

lize Optuna, a hyperparameter optimization framework, to automate the tuning process by allowing it to explore the parameter space and determine desirable hyperparameters for each algorithm. At the top of each notebook, we define the environment to use according to a string parameter, where the string is one of the environments listed in table 1.

```
# === Create wrapped evaluation env ===
def make_env(str_env=None):
    def _init():
        if str_env is None or
            str_env == "highway":
            env_id = "highway-better-v1"
        elif str_env == "intersection":
            env_id = "intersection-v1"
        elif str_env == "racetrack":
            env_id = "racetrack-v0"
        else:
            raise ValueError("...")

        env = gym.make(env_id, ...)
        return Monitor(env)
    return _init
```

Note that tuning an algorithm will generate a **tensorboard_logs** directory, which can be used to visualize the training process and the performance of the agent during tuning. Additionally, the best tuned models are saved to **trained_models** where the model weights can be loaded for evaluation.

Then, we call our code-block to perform hyperparameter tuning, passing the environment string to the objective function that optuna will use to create the environment for each trial. This way, we replicate the same hyperparameter tuning process for each environment and algorithm combination.

For each trial, we train the agent for a fixed number of steps (10,000-40,000 dependent on the algorithm) and evaluate its performance on the environment. We also incorporate early stopping to prune unpromising trials, which helps in efficiently searching through the hyperparameter space. This proved effective, particularly for SAC where the training time became significant compared to the other

algorithms (we also tune SAC for substantially longer timesteps). Implementation of pruning can be found in the OptunaCallback, which is passed to each trial’s learn function call.

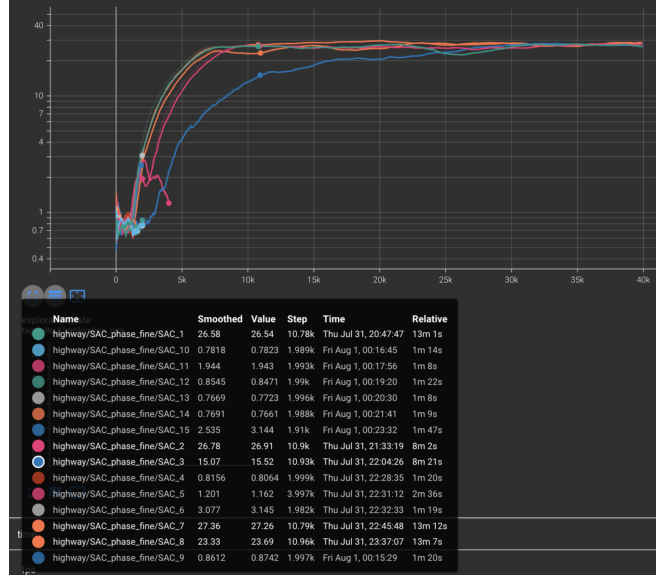


Fig. 1. Detailed plot (SAC agent on highway environment) demonstrating a typical run for tuning, notice the pink line that gets pruned early near step 5,000.

3.1 Tuning DQN

We train DQN on the highway and intersection environments, as they are defined for discrete action spaces. We perform 15 trials, optimizing over the following hyperparameters:

```
gamma = trial.suggest_float("gamma", 0.85, 0.999)
learning_rate = trial.suggest_float(
    "learning_rate", 1e-5, 1e-3, log=True)
exploration_fraction = trial.suggest_float(
    "exploration_fraction", 0.1, 0.5)
net_arch = trial.suggest_categorical(
    "net_arch", [[64, 64], [128, 128], [256, 256]])
batch_size = trial.suggest_categorical(
    "batch_size", [32, 64, 128, 256])
learning_starts = trial.suggest_categorical(
    "learning_starts", [1000, 2000, 5000])
```

Note that `gamma` is the discount factor, which determines how much future rewards are considered in the value function; the `learning_rate` controls the step size in the optimization process; the `exploration_fraction` is the fraction of time the agent explores the environment rather than exploiting its learned policy; the `net_arch` defines the structure of the neural network used to approximate the q-values; the `batch_size` is the number of samples used in each training step.

We also use a replay buffer size of 15,000 – we found various implementations of DQN on the highway environment, and Atari domain using this size for the replay buffer – which only fully comes into effect after the agent has

been trained for 15,000 steps. To ensure we perform tuning in a timely manner, we limit the number of steps to 10,000 for each trial.

3.2 Tuning PPO

For each subsequent algorithm, we split the tuning process into two parts: coarse and fine-grained tuning.

Since PPO works for both discrete and continuous action spaces, we tune it on all three environments, but also tuning a discrete version of the intersection environment (intersection-v0) to compare with the continuous version (intersection-v1). We use the following coarse grained hyperparameters:

```
gamma = trial.suggest_float(
    "gamma", 0.85, 0.999)
learning_rate = trial.suggest_float(
    "learning_rate", 1e-5, 1e-3, log=True)
net_arch = trial.suggest_categorical(
    "net_arch", [[64, 64],
                  [128, 128],
                  [256, 256]])
```

and then fine-tune the following hyperparameters based on the results of the coarse tuning:

```
"entropy_coef": trial.suggest_float(
    "entropy_coef", 1e-4, 0.05),
"clip_range": trial.suggest_float(
    "clip_range", 0.1, 0.3),
"gae_lambda": trial.suggest_float(
    "gae_lambda", 0.8, 0.99),
"vf_coef": trial.suggest_float(
    "vf_coef", 0.3, 0.9),
"max_grad_norm": trial.suggest_float(
    "max_grad_norm", 0.3, 1.0),
"n_steps": trial.suggest_categorical(
    "n_steps", [64, 128, 256]),
"batch_size": trial.suggest_categorical(
    "batch_size", [32, 64, 128])
```

Note that the `entropy_coef` is used to encourage exploration by adding an entropy term to the loss function, which helps prevent premature convergence to suboptimal policies; the `clip_range` is used to limit the change in the policy during training, helping to stabilize the training process; the `gae_lambda` is used in Generalized Advantage Estimation to compute the advantage function, which helps in reducing variance in the policy gradient estimates; the `vf_coef` is the coefficient for the value function loss in the overall loss function; the `max_grad_norm` is used to clip the gradients during training to prevent exploding/vanishing gradients;

Similar to DQN, we limit the number of steps to 10,000 for each trial to ensure timely tuning.

3.3 Tuning SAC

We tune SAC on the continuous action space variants of each environment, with a trial executing 25,000 - 40,000 steps (Note that we trained SAC on DQN initially for

40,000 steps on the highway environment, but found this to take too long, so we reduced the number of steps to 25,000 for the remaining environments). We perform coarse tuning on the following hyperparameters:

```
learning_rate = trial.suggest_float(
    "learning_rate", 1e-5, 1e-3, log=True)
gamma = trial.suggest_float(
    "gamma", 0.9, 0.999)
net_arch = trial.suggest_categorical(
    "net_arch", [[64, 64],
                  [128, 128],
                  [256, 256]])
learning_starts = trial.
suggest_categorical(
    "learning_starts", [1_000, 2_000])
```

and then fine-tune the following hyperparameters based on the results of the coarse tuning:

```
batch_size = trial.suggest_categorical(
    "batch_size", [16, 32, 64, 128, 256])
tau = trial.suggest_float(
    "tau", 0.005, 0.02)
train_freq = trial.suggest_categorical(
    "train_freq", [1, 8, 16, 64])
gradient_steps = trial.
suggest_categorical(
    "gradient_steps", [1, 4, 8])
ent_coef = trial.suggest_categorical(
    "ent_coef", ["auto", "auto_0.1"])
use_sde = trial.suggest_categorical(
    "use_sde", [False, True])
```

Note that `tau` is the soft update coefficient for the target network when updating the policy and value networks; `use_sde` determines whether to use state dependent exploration, which can help in improving exploration in certain domains.

3.4 Tuning TRPO

Finally, we tune TRPO on the racetrack environment, where each trial is executed for 10,000 steps.

We perform coarse tuning on the following hyperparameters:

```
gamma = trial.suggest_float(
    "gamma", 0.85, 0.999)
learning_rate = trial.suggest_float(
    "learning_rate", 1e-5, 1e-3, log=True)
net_arch = trial.suggest_categorical(
    "net_arch", [[64, 64],
                  [128, 128],
                  [256, 256]])
batch_size = trial.suggest_categorical(
    "batch_size", [32, 64, 128])
```

and then fine-tune the following hyperparameters based on the results of the coarse tuning:

```
cg_max_steps = trial.suggest_int(
    "cg_max_steps", 5, 30)
```

```
cg_damping = trial.suggest_float(
    "cg_damping", 0.0005, 0.2)
target_kl = trial.suggest_float(
    "target_kl", 0.005, 0.2)
```

Note that `cg_max_steps` is the maximum number of steps for the conjugate gradient algorithm used in TRPO, higher values can lead to more accurate estimates of the policy gradient but increase computation time; `cg_damping` is a damping factor added to the matrix used by the conjugate gradient algorithm to improve numerical stability; `target_kl` is the target KL divergence between the old and new policies, which influences when policies are updated during training.

4 Training and Evaluation

Due to time constraints, we evaluated the agents immediately after tuning, without performing additional training runs beyond the best-found parameters from the Optuna studies.

4.1 Intersection Environment

As mentioned early, the destination in the intersection env (discrete, and continuous) is set to 'o1', which corresponds to a left turn at the junction. We also note that we turned off lateral movement from the action set in the discrete environment. In theory, the agent should be able to navigate through the intersection by keeping its lane and adjusting its speed to avoid collisions.

Our evaluation revealed that both the SAC and continuous PPO agents performed poorly in this domain. These agents often exhibited erratic behavior, such as spinning off the road or oscillating between lanes. In contrast, DQN showed strong performance, reaching the destination 49 % of the time, over the span of 100 episodes (note here that this number should be a bit higher since other vehicles colliding in the intersection would terminate the episode).

4.1.1 PPO Comparison

To investigate this discrepancy, we directly compared PPO agents trained on `intersection-v0` (discrete) and `intersection-v1` (continuous). The discrete PPO agent, like DQN, successfully navigated the intersection and executed safe turns.

Comparing the discrete PPO agent to the DQN agent, we found that the PPO agent had a success rate 54 % of the time compared to DQN's 49 %. Though the results indicate similar performance, we notice that PPO deliberately slows down when approaching other vehicles, whereas DQN seems to try to fly through the intersection by chance. Graphically, the tuning results are similar between the two agents, with both achieving relatively similar rewards at the end of tuning.

We hypothesize that the discrete action space provides a structural bias that simplifies the learning problem. In this setting, successful navigation can often be achieved through a limited set of predefined speed adjustments. By contrast, continuous agents must learn fine-grained steer-

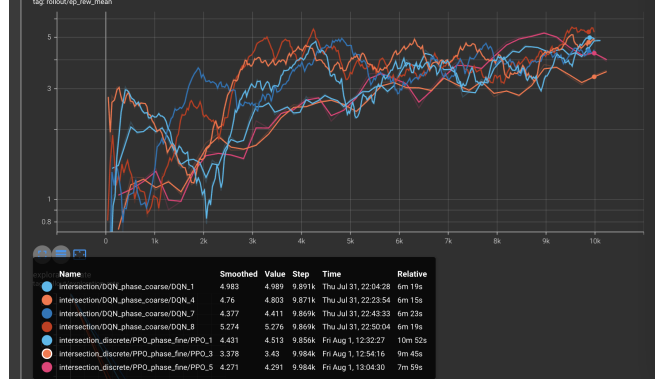


Fig. 2. PPO and DQN performance on the intersection environment (log scale).

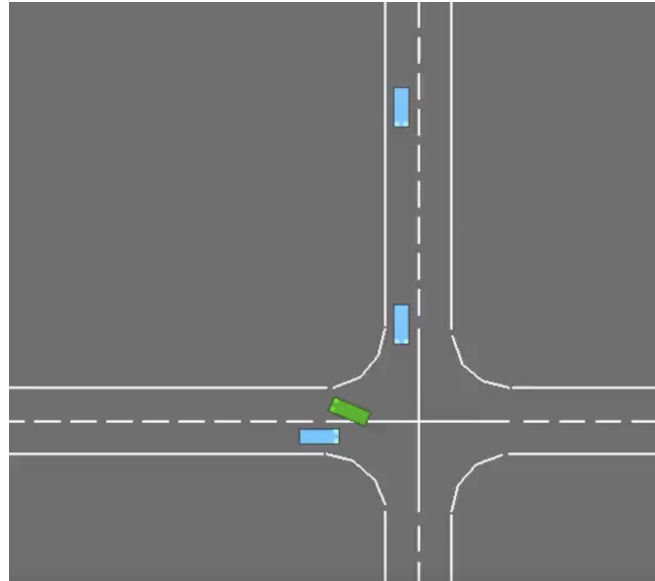


Fig. 3. Visualization of the discrete PPO agent navigating through an intersection. The agent successfully executes a left turn while avoiding collisions with other vehicles.

ing and throttle control, a significantly harder problem requiring more training data and stability. With more training, continuous agents might eventually learn suitable control policies, but within our limited training budget, discrete actions were more effective.

4.2 Highway Environment

The highway environment posed unique challenges, including dense traffic, variable speeds, and frequent lane changes. Here, the performance of all agents was relatively weak.

The DQN agent tended to accelerate aggressively into other vehicles, terminating episodes early. This may reflect a failure to balance the sparse progress reward with penalties for collisions or insufficient exploration. PPO and SAC agents exhibited the same “jittering” seen in the intersection environment, frequently oscillating between acceleration and deceleration and often yielding to surrounding traffic until time ran out.

We speculate that the high-level goal of highway driving—efficiently overtaking vehicles while maintaining safety—is difficult to express through the existing reward function. Without explicit guidance, the agents may misinterpret reward signals, leading to overly cautious or overly aggressive behavior. Our findings suggest that discrete PPO could be a more suitable choice in this setting, but that reward shaping and longer training are likely necessary to achieve reliable driving behavior. Since the reward for collisions is -1 , as training progresses, the agents learn to avoid collisions at the cost of interesting driving behavior – choosing to exploit suboptimal policies.

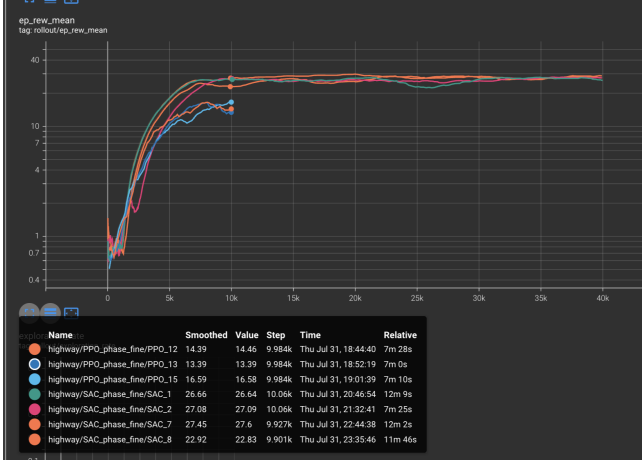


Fig. 4. Log scale performance of PPO and SAC on the highway environment. Though we would expect SAC to outperform PPO, evaluation results on the highway environment show that SAC has found a suboptimal policy and doesn’t exhibit interesting driving behavior.

4.3 Racetrack Environment

In the racetrack environment, agents are required to follow a circuit with curves and other vehicles. PPO performed reasonably well after 10,000 steps of training: it was able to follow the track and complete laps with occasional lane changes. However, it exhibited some instability, such as drifting off-track or jittering in steering. It also struggled with collision avoidance, especially when tailgating another vehicle.

TRPO exhibited similar problems but was more prone to veering off-road, cutting across grass, and even colliding head-on with other cars. This may be due to the stricter policy constraints enforced by TRPO, which can slow learning and lead to suboptimal exploration in highly dynamic environments.

SAC stood out as the best-performing agent in this domain. Its stochastic policy and entropy-regularized objective allowed it to explore effectively and learn robust control. The SAC agent followed the track smoothly, showed minimal jitter, and was capable of sophisticated behaviors—such as switching to the inside lane on curves and returning to the outside afterward. It also handled overtaking maneuvers well, adapting dynamically to traffic and maintaining consistent progress.

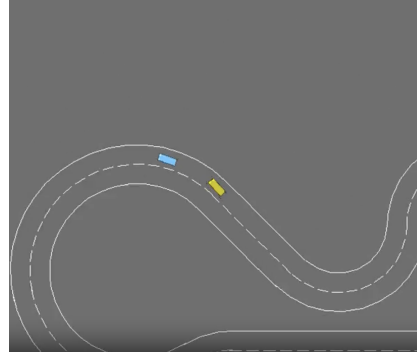


Fig. 5. SAC agent directly following the track behind another vehicle.

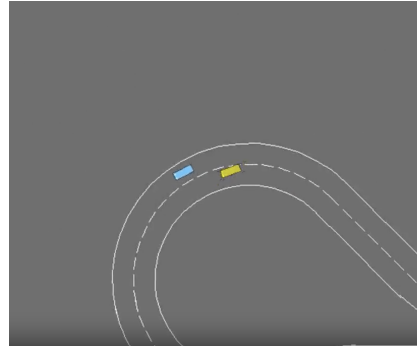


Fig. 6. SAC agent preparing to overtake another vehicle on the track.

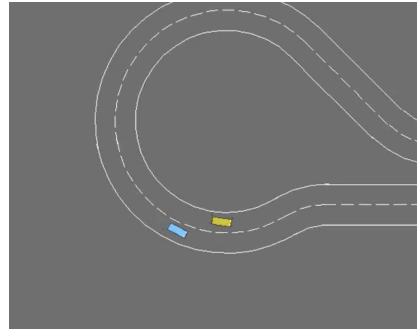


Fig. 7. SAC agent successfully overtaking another vehicle on the track.

Overall, SAC demonstrated strong performance in continuous control tasks with spatial complexity, justifying its design for continuous action spaces. PPO offered reasonable generalization across tasks, while DQN and discrete PPO proved most effective in structured, rule-based environments like intersections.

5 Conclusion

This assignment explored the application of various DRL algorithms in the HighwayEnv framework, focusing on highway, intersection, and racetrack environments. We feel confident that the results demonstrate the strengths and weaknesses of each algorithm in different scenarios, highlighting the importance of environment characteristics and

action space in DRL performance. Future work could involve further tuning, training, reward shaping, and exploring alternative architectures to improve performance across all environments.

While our results are not promising for the highway environment, we demonstrated the effectiveness of such algorithms in the intersection and racetrack environments; and overall feel more confident in further exploration of DRL algorithms on additional domains.

6 Appendix

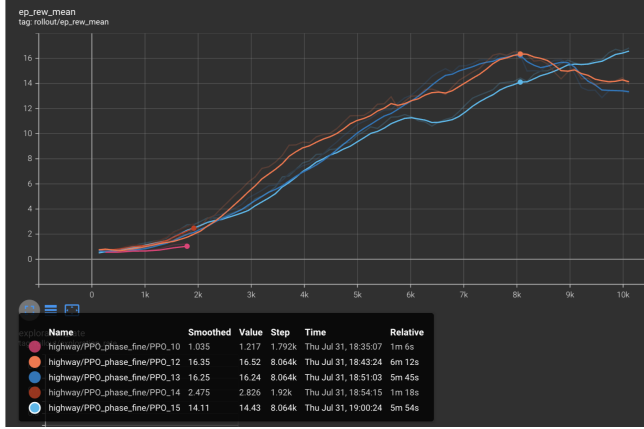


Fig. 8. PPO tuning performance on the highway environment (log scale).

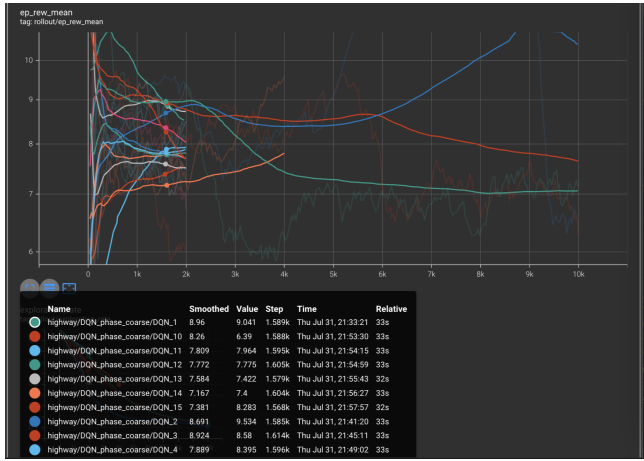


Fig. 9. DQN tuning performance on the highway environment (log scale).

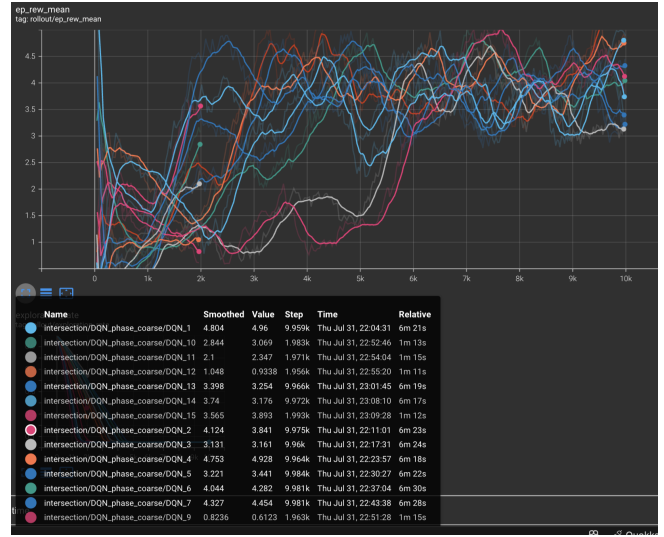


Fig. 10. DQN tuning performance on the intersection environment.

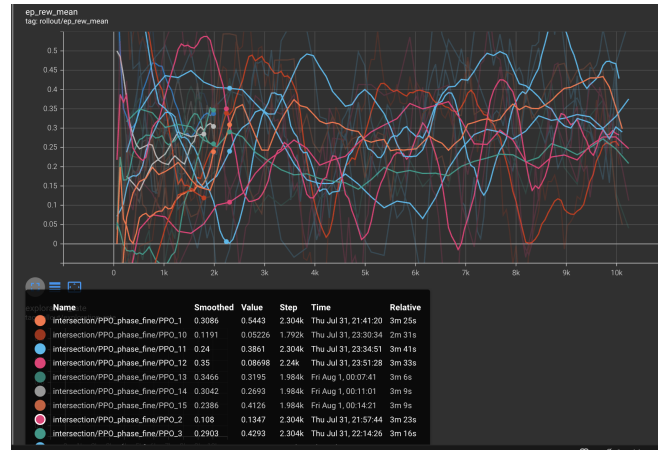


Fig. 11. PPO tuning performance on the intersection environment.

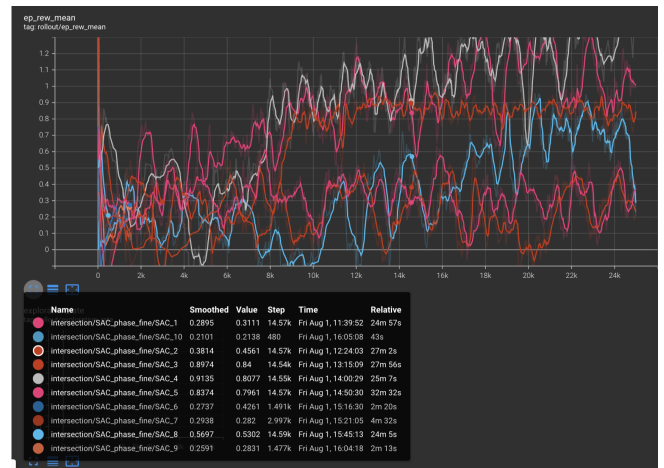


Fig. 12. SAC tuning performance on the intersection environment.

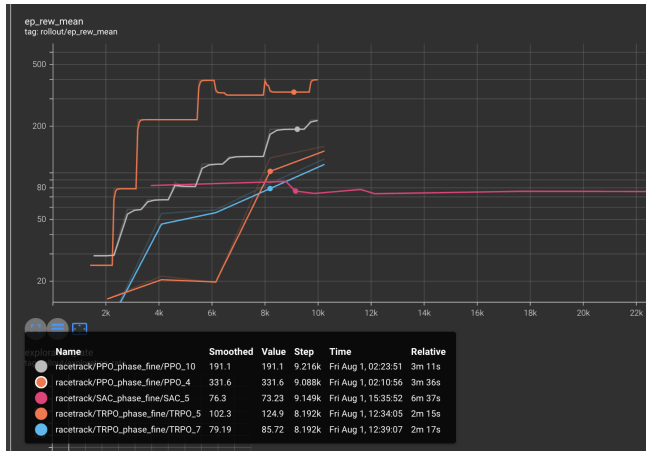


Fig. 13. Comparison of all agents on the racetrack environment. The SAC agent has the lowest reward at the end of tuning, yet it performs the best in terms of driving behavior and control.